desenvolvimento-verificavel/**SKILL.md**

---

name:

DESENVOLVIMENTO VERIFICÁVEL

COM AGENTES DE IA

description: Skills, processo e evidência, na prática com o superpowers

author: **Regis Araujo Melo**



QUEM APRESENTA

REGIS ARAUJO MELO

Engenheiro iOS Sênior

Desenvolvedor iOS sênior com mais de 20 anos em tecnologia, com foco em apps nativos para os setores financeiro, varejo e mobilidade. Arquitetura modular, segurança, acessibilidade e performance, evolução de código legado e entrega contínua.

LinkedIn
linkedin.com/in/regisaraujomelo

ROTEIRO

60 MINUTOS, 25 DE PRÁTICA

0:00–0:15

Conceitos

o problema, a anatomia de um agente, skills, o fluxo mínimo e SDD

ENTREGA

um vocabulário comum para o resto da hora

0:15–0:40

Laboratório

esclarecer, planejar e testar primeiro, no seu repositório

ENTREGA

design aprovado, plano, teste vermelho → verde, commit local

0:40–1:00

Retrospectiva e limites

o que as evidências mostraram, o que vem depois e perguntas

ENTREGA

as evidências da turma, lidas no chat

O PROBLEMA

RÁPIDO NÃO QUER DIZER VERIFICÁVEL

Sem processo, um agente pode — por exemplo, na lista de bancos do laboratório:

Implementar antes de esclarecer o requisito

Fez a busca só por nome; ninguém perguntou sobre o código do banco.

Escrever o teste depois, só para passar

O teste copia a saída do código, com a ordem errada junto.

Dizer “pronto” sem evidência

Respondeu “pronto!”, mas não rodou o teste.

Resultado: a revisão fica toda no fim, e cara.

E se o processo viesse junto com o agente?

CONCEITO

AGENTE = MODELO + HARNESS



Modelo (LLM)
Gera texto e decide o próximo passo. Sozinho, não age.

Contexto
O que o modelo vê agora. Tem limite.

Harness
Roda o loop, pede a sua permissão e carrega as skills.

Ferramentas · MCP
As mãos do agente; MCP é o padrão para conectar serviços externos.

No chat, **o harness é você**: você copia, cola e executa.

No laboratório, o harness é o Copilot CLI, e as skills dizem como ele trabalha.

CONCEITO

SKILL É UM PROCESSO EM ARQUIVO

Uma pasta com um `SKILL.md`: nome, descrição e instruções. Entra em uso pela descrição ou quando você a cita (`/nome`).

SEMPRE NO CONTEXTO

```
---
name: brainstorming
description: Use antes de qualquer
  trabalho criativo…
---
```

SÓ QUANDO É USADA

```
# Brainstorming
1. Explore o contexto do projeto
2. Faça uma pergunta por vez
3. Proponha 2-3 abordagens…
```

Prompt

O que você pede agora. Some quando a conversa acaba.

Ferramenta · MCP

O que o agente consegue fazer: ler arquivo, rodar comando, acessar um serviço.

Skill

Como o agente deve fazer: um método versionado e revisável como código.

O FLUXO MÍNIMO

QUATRO SKILLS PARA O FLUXO MÍNIMO

01

Esclarecer

`brainstorming`

requisito e design aprovados

EVITA

implementar antes de esclarecer o requisito

02

Planejar

`writing-plans`

plano em passos pequenos

EVITA

a revisão toda no fim

03

Testar primeiro

`test-driven-development`

teste vermelho, depois verde

EVITA

o teste escrito depois, só para passar

04

Verificar

`verification-before-completion`

“pronto” só com evidência

EVITA

o “pronto” sem evidência

O pacote tem 14 skills. Hoje usamos estas quatro; o catálogo completo está no apêndice.

DEFINIÇÃO

CONFIÁVEL = EVIDÊNCIA OBSERVÁVEL

ANTES

esclarecer · planejar

- **Requisito esclarecido**
  docs/superpowers/specs/
- **Plano revisável**
  docs/superpowers/plans/

DURANTE

testar primeiro

- **Teste falhando antes do código**
  a saída vermelha do teste
- **Teste passando depois**
  a saída verde do teste

DEPOIS

verificar

- **Diff revisado por você**
  git diff antes do commit
- **Resultado verificado por você**
  o teste rodado por você, não pelo agente

Sem essas evidências, é só código que parece certo.

SPEC-DRIVEN DEVELOPMENT

O QUE É SDD

Desenvolvimento guiado por especificação: antes do código, você escreve o que o sistema deve fazer. A spec vira a referência para planejar, implementar e verificar.

Código primeiro	
FLUXO	pedido → código
LACUNAS	o agente preenche sozinho
PRONTO	parece certo

Spec primeiro	
FLUXO	pedido → spec → código
LACUNAS	viram perguntas, e você decide antes
PRONTO	cumpre a spec

UMA SPEC TEM

- objetivo
- requisitos
- critérios de aceite
- fora do escopo

SPEC-DRIVEN DEVELOPMENT

SDD: A SPEC VEM PRIMEIRO

Cada etapa gera um artefato que você revisa antes da próxima.

01
spec
o quê e o porquê: requisitos e critérios de aceite

→

02
plano
o como: decisões técnicas a partir da spec

→

03
tarefas
passos pequenos e verificáveis

→

04
código
implementado e verificado contra a spec

Definição: README do github/spec-kit, consultado em 19/09/2026.

SPEC-DRIVEN DEVELOPMENT

SPEC KIT: SDD
COMO FERRAMENTA

Mesmo método, outra ferramenta. Hoje usamos o **superpowers**; o Spec Kit é uma alternativa.

`/speckit-constitution`	PRINCÍPIOS	regras do projeto que toda spec respeita
`/speckit-specify`	SPEC	a spec: o quê e o porquê
`/speckit-plan`	PLANO	o plano técnico: o como
`/speckit-tasks`	TAREFAS	tarefas a partir do plano
`/speckit-implement`	CÓDIGO	implementa as tarefas; repete com **`/speckit-converge`** até convergir

GitHub Spec Kit (github/spec-kit), integração padrão com o GitHub Copilot. Comandos conforme o README em 19/09/2026: confira a versão.

SPEC-DRIVEN DEVELOPMENT

O EXEMPLO, ETAPA A ETAPA

SPEC

docs/superpowers/specs/

OBJETIVO	escolher o banco de origem numa lista
REQUISITOS	ordena por nome, sem diferenciar maiúsculas nem acentos; busca no nome (contém) ou no código (prefixo)
ACEITE	“credito” encontra “Crédito”; “00” encontra “001”; busca vazia devolve todas
FORA	tela e rede

PLANO

docs/superpowers/plans/

DECISÃO	uma função pura `filterAndSort(institutions, query)`
POR QUÊ	sem tela e sem rede, o teste roda em segundos
ACENTOS	comparar textos normalizados (sem acento, minúsculos)

TAREFAS

uma por teste

1. **Ordenar por nome:** três bancos fora de ordem → ordenados
2. **Filtrar por nome:** “brasil” → “Banco do Brasil”
3. **Código e busca vazia:** “00” → “001”; “” → todas

CÓDIGO

tarefa 1, teste primeiro

DADO	[Itaú, Bradesco, Caixa]
ESPERA	[Bradesco, Caixa, Itaú]
✗ VERMELHO	`filterAndSort` não existe
✓ VERDE	depois da implementação

Trechos resumidos. No laboratório, o agente escreve esses arquivos e você revisa.

BRAINSTORMING NA PRÁTICA

O AGENTE PERGUNTA ANTES DE CODAR

É assim que a spec nasce: o `brainstorming` pergunta, você responde, e cada resposta vira requisito. A saída é um design curto, **aprovado por você**.

```
agente › Como a lista deve ser ordenada?
você   › Por nome, sem diferenciar maiúsculas.  (1)
agente › A busca vale para nome, código ou os dois?
você   › Os dois. Código só por prefixo.  (2)
agente › “credito” deve encontrar “Crédito”?
você   › Sim, sem diferenciar acentos.  (3)
```

Exemplo ilustrativo, não é saída real do agente.

→

docs/superpowers/specs/ — DESIGN APROVADO

1. Ordena por nome, sem diferenciar maiúsculas
2. Busca: nome contém o texto, ou código começa com ele
3. Ordem e busca ignoram acentos

\+ Busca vazia devolve todas
PROPOSTO PELO AGENTE

SPEC-DRIVEN DEVELOPMENT

O FLUXO DE HOJE JÁ É SDD

NO SDD	NO FLUXO MÍNIMO	NO LABORATÓRIO
spec	**Esclarecer** `brainstorming`	ETAPA 1 · 8 MIN design aprovado
plano e tarefas	**Planejar** `writing-plans`	ETAPA 2 · 4 MIN plano com até 3 tarefas
código	**Testar primeiro** `test-driven-development`	ETAPA 3 · 11 MIN vermelho, depois verde
verificação	**Verificar** `verification-before-completion`	ETAPA 3 você roda o teste

TAREFA PRÉVIA

PRONTO PARA COMEÇAR?

1 Login — Abra o `copilot` e entre com `/login`.

2 Instalar

```
npx skills add obra/superpowers -g -a github-copilot
-s brainstorming writing-plans test-driven-development verification-before-completion
```

3 Conferir — De volta ao `copilot`: `/skills list` mostra as **quatro** skills.

4 Repositório — O build da sua stack roda. Os comandos estão no guia.

Faltou algo? Os pré-requisitos e o guia estão no chat: copie de lá, não da tela.

Tudo certo? Escreva pronto no chat.

Sem ambiente? Forme dupla ou acompanhe a tela do apresentador.

ANTES DE COMEÇAR

REGRAS DO LABORATÓRIO

Branch local, sem push

`workshop/seu-nome`, só na sua máquina. Nunca aprove um `git push`.

Aprove comando a comando

Leia cada comando antes de permitir. Nunca “permitir tudo”.

Nada sensível

Não cole segredos, tokens nem dados de clientes. Não peça para ler `.env` nem `local.properties`.

ETAPA 0 · 1 MIN · JUNTOS

```
# na pasta do repositório
$ git switch -c workshop/seu-nome
$ copilot
```

A SEGUIR · 25 MIN

1	**Esclarecer**	8 min
2	**Planejar**	4 min
3	**Vermelho → verde**	11 min

Um checkpoint no chat em cada etapa.

LABORATÓRIO · ETAPA 1 DE 3

8 min

ESCLARECER

1. No `copilot`, cole o pedido **da sua stack** (iOS, Android ou backend), do guia.
2. Responda às perguntas: no máximo 3, respostas curtas.
3. **Confira e aprove** o design: ordem, busca e acentos estão lá?

A HISTÓRIA E O RECORTE

Como cliente que quer trazer investimentos de outro banco, **quero** escolher o banco de origem numa lista, **para** iniciar a portabilidade sem digitar dados do banco. **Recorte:** uma função pura (sem tela nem rede) que ordena por nome e filtra por nome ou código; busca vazia devolve todas.

{"code": "001", "institutionType": "1", "name": "Banco do Brasil", "cnpj": "00000000000191"}

Checkpoint no chat: a pergunta mais útil que o agente fez. Passou de 8 min? Use o design pronto do guia.

LABORATÓRIO · ETAPA 2 DE 3

4 min

PLANEJAR

1. Na mesma sessão, peça o plano com `/writing-plans` (pedido da etapa 2).
2. No máximo **3 tarefas**.
3. **Confira:** cada tarefa começa por um teste, a Tarefa 1 é ordenar por nome, e nada sai do recorte (tela, rede).

UM BOM PLANO SE PARECE COM ISTO

`docs/superpowers/plans/`

1	**Ordenar por nome**	TESTE: três bancos fora de ordem → ordenados
2	**Filtrar por nome**	TESTE: “brasil” → “Banco do Brasil”
3	**Código e busca vazia**	TESTE: “00” → “001”; “” → todas

Checkpoint no chat: quantas tarefas o seu plano tem. Travou? Use o plano pronto do guia.

LABORATÓRIO · ETAPA 3 DE 3

11 min

VERMELHO, DEPOIS VERDE

1. Cole o pedido da etapa 3: só a Tarefa 1, com `/test-driven-development`.
2. O agente escreve o teste e **para**. **Você** roda: vermelho.
3. O agente implementa o mínimo. **Você** roda de novo: verde.
4. Revise com `git diff` e faça commit só local.

```
✗ FALHOU
ordena por nome
filterAndSort não existe
```



```
✓ PASSOU
ordena por nome
1 teste, 0 falhas
```

Ilustrativo: a saída real depende da sua stack.

Checkpoint no chat: a saída do teste verde. Não terminou? O guia mostra como continuar.

RETROSPECTIVA

O QUE AS EVIDÊNCIAS MOSTRARAM

ETAPA 1 · ESCLARECER

A pergunta mais útil

Que requisito ela esclareceu antes do código?

EVIDÊNCIA
requisito esclarecido

ETAPA 2 · PLANEJAR

Quantas tarefas?

Alguém precisou cortar?

EVIDÊNCIA
plano revisável

ETAPA 3 · VERMELHO → VERDE

Quantos chegaram ao verde?

Pela saída colada no chat.

EVIDÊNCIA
teste falhando, depois passando

Onde travou? Travar também é dado: mostra onde o processo protege.

PARA A TURMA

Dos três problemas do começo, qual o processo evitou hoje?

- implementar antes de esclarecer
- teste escrito só para passar
- “pronto” sem evidência

ESCALA, REVISÃO E ENTREGA

DEPOIS DO PRIMEIRO TESTE

Você fez a Tarefa 1. As tarefas 2 e 3 seguem o mesmo método, e a entrega ganha revisão.

01

`subagent-driven-development`

seguir o plano tarefa a tarefa

sem subagentes: `executing-plans`

→

02

`requesting-code-review`

revisão independente antes de integrar

→

03

`finishing-a-development-branch`

merge, PR ou descarte, com os testes verdes

Essas três não vieram na instalação de hoje. O comando com o fluxo completo está no apêndice (**slide 33**).

COM OS PÉS NO CHÃO

LIMITES E CONTROLES

Custa tempo e tokens

Use o fluxo completo onde há dúvida ou risco. Correção de uma linha não precisa.

Skill de terceiros

É instrução para o seu agente. Revise o **`SKILL.md`** e os scripts antes de instalar.

A skill orienta, não garante

O agente pode pular etapas. Por isso a evidência é conferida por você, e o PR continua com a sua assinatura.

OS CONTROLES DO LABORATÓRIO VALEM NO DIA A DIA

- ✓ branch local, sem push
- ✓ aprovar comando a comando
- ✓ nada sensível
- ✓ rodar o teste você mesmo

DEPOIS DO WORKSHOP

PRÓXIMO PASSO: UMA HISTÓRIA REAL

1. **Termine o laboratório** com a seção “Como continuar” do guia.
2. Use o fluxo numa **história pequena** desta sprint.
3. Traga as **seis evidências** na próxima reunião.

AS SEIS EVIDÊNCIAS

- [ ] requisito esclarecido
- [ ] plano revisável
- [ ] teste vermelho
- [ ] teste verde
- [ ] diff revisado
- [ ] resultado verificado por você

Skill não torna o agente infalível;
torna o trabalho dele verificável.

Perguntas?

APÊNDICE

MATERIAL DE APOIO

Para usar se sobrar tempo ou surgir uma pergunta.

1 **O mapa das 14 skills** e os quatro grupos 25-29

2 **brainstorming**, **writing-plans** e **executing-plans** em detalhe 30-32

3 **O fluxo completo** em sete etapas 33

4 **obra, skills.sh** e skills por plataforma 34-37

O MAPA DO SUPERPOWERS

14 SKILLS, 4 MOMENTOS DO TRABALHO

Hoje usamos quatro. O fluxo completo usa oito. As outras ficam para quando você precisar.

01 Pensar antes de codar

- brainstorming
- writing-plans
- executing-plans

02 Qualidade e disciplina

- test-driven-development
- systematic-debugging
- verification-before-completion

03 Escala com agentes

- subagent-driven-development
- dispatching-parallel-agents
- using-git-worktrees

04 Revisão e entrega

- requesting-code-review
- receiving-code-review
- finishing-a-development-branch
- writing-skills
- using-superpowers

instalada hoje · fluxo completo · slide 33 · outras

01

PENSAR ANTES DE CODAR

Objetivo: chegar ao código com spec e plano aprovados.

SEM A SKILL	COM A SKILL, O AGENTE…
`brainstorming` instalada hoje `Etapa 1 · design aprovado` Pedido vago vira código errado.	faz perguntas uma a uma, propõe 2 ou 3 abordagens e só avança com um design aprovado.
`writing-plans` instalada hoje `Etapa 2 · plano com até 3 tarefas` Tarefa grande vira improviso.	quebra o trabalho em passos pequenos, com arquivos, testes e critério de pronto.
`executing-plans` outra O plano é esquecido no meio do caminho.	executa o plano em lotes, com pontos de revisão. É a alternativa ao `subagent-driven-development` quando não há subagentes.

02

QUALIDADE E DISCIPLINA

Objetivo: cada afirmação acompanhada de evidência.

SEM A SKILL	COM A SKILL, O AGENTE…
test-driven-development instalada hoje Etapa 3 · vermelho, depois verde	
Teste escrito depois só confirma o código.	segue vermelho, verde, refatorar: o teste falha primeiro.
systematic-debugging outra	
O bug é “consertado” no chute.	reproduz, isola e encontra a causa raiz antes de corrigir.
verification-before-completion instalada hoje Etapa 3 · você roda o teste	
“Pronto!” sem prova.	só declara pronto com evidência: teste rodado, saída conferida.

03

ESCALA COM AGENTES

Objetivo: paralelizar com revisão entre tarefas.

SEM A SKILL	COM A SKILL, O AGENTE…
`subagent-driven-development` `fluxo completo`	
Um agente só se perde num contexto longo.	entrega cada tarefa do plano a um subagente novo, com revisão entre tarefas, onde houver suporte.
`dispatching-parallel-agents` `outra`	
Problemas independentes resolvidos em fila.	dispara vários agentes em paralelo, um por problema.
`using-git-worktrees` `fluxo completo`	
Tarefas paralelas se atropelam no mesmo diretório.	isola cada frente de trabalho na sua própria worktree.

04

REVISÃO E ENTREGA

Objetivo: revisar, integrar e registrar o que se aprendeu.

SEM A SKILL	COM A SKILL, O AGENTE…
`requesting-code-review` fluxo completo O agente revisa o próprio trabalho, com o próprio viés.	pede uma revisão independente antes de integrar.
`receiving-code-review` outra Aceita toda sugestão sem pensar.	avalia cada sugestão com rigor técnico e contesta quando necessário.
`finishing-a-development-branch` fluxo completo Branch pronta e esquecida.	apresenta opções claras: merge, PR ou limpeza.
`writing-skills` outra O time aprende algo e o conhecimento se perde.	transforma o aprendizado numa nova skill, testada.
`using-superpowers` outra O agente não sabe que as skills existem.	procura e usa a skill certa antes de agir. É a porta de entrada.

PENSAR ANTES DE CODAR · EM DETALHE

brainstorming

QUANDO USAR

Antes de qualquer trabalho criativo: uma feature nova, um componente, uma mudança de comportamento.

REGRA DE OURO

> **Nenhum código antes de um design aprovado. Vale até para o que "parece simples".**

O QUE GERA

`docs/superpowers/specs/AAAA-MM-DD-tema-design.md`

Na prática: na lista de bancos, o agente pergunta a ordem, como buscar e se ignora acentos. Você aprova o design, e ele ainda propõe que a busca vazia devolva todas.

O PROCESSO

1. **Explora o contexto:** arquivos, docs e commits recentes.
2. **Pergunta uma coisa por vez,** de preferência com múltipla escolha.
3. **Propõe 2 ou 3 abordagens** com prós, contras e uma recomendação.
4. **Apresenta o design em partes** e pede aprovação de cada uma.
5. **Escreve a spec e faz commit,** depois revisa o texto e pede a revisão de quem pediu.
6. **Passa a vez para** `writing-plans`.

PENSAR ANTES DE CODAR · EM DETALHE

writing-plans

QUANDO USAR

Com uma spec aprovada em mãos, antes de tocar no código.

REGRA DE OURO

> **Escrever para um dev que não conhece o projeto: nada de “TODO” ou “implementar depois”.**

O QUE GERA

`docs/superpowers/plans/AAAA-MM-DD-feature.md`

Na prática: a lista de bancos vira três tarefas, cada uma com o seu teste: ordenar por nome, filtrar por nome, código e busca vazia. No laboratório, limitamos a 3 tarefas para caber em 4 minutos.

O PROCESSO

1. **Mapeia os arquivos** e a responsabilidade de cada um.
2. **Quebra em tarefas pequenas:** cada passo leva de 2 a 5 minutos.
3. **Em cada tarefa:** teste que falha, implementação, teste passando, commit.
4. **Escreve tudo por extenso:** caminhos exatos, código completo, comando e saída esperada.
5. **Revisa o plano contra a spec:** cobertura, pendências e nomes consistentes.
6. **Oferece a execução:** com `subagent-driven-development` ou `executing-plans`.

PENSAR ANTES DE CODAR · EM DETALHE

executing-plans

QUANDO USAR

Quando há um plano escrito para executar com pontos de revisão. Com subagentes disponíveis, prefira `subagent-driven-development`. Não está no comando do slide 33: acrescente `executing-plans` ao `-s`.

REGRA DE OURO

> **Travou? Para e pergunta. Não adivinha e não força o caminho.**

O QUE GERA

Código commitado numa branch, pronto para integrar.

Na prática: depois do workshop, as tarefas 2 e 3 da lista de bancos. Se o teste de “brasil” falhar sem motivo claro, o agente para e pergunta.

O PROCESSO

1. **Prepara um espaço isolado** com `using-git-worktrees`.
2. **Lê o plano com olhar crítico** e levanta dúvidas antes de começar.
3. **Executa tarefa por tarefa:** segue os passos à risca e roda as verificações.
4. **Para no primeiro bloqueio:** dependência faltando, teste falhando, instrução confusa.
5. **Fecha com** `finishing-a-development-branch`: merge, PR ou limpeza.

JUNTANDO TUDO

O FLUXO COMPLETO

01 brainstorming › 02 writing-plans › 03 using-git-worktrees › 04 subagent-driven-development + test-driven-development › 05 requesting-code-review › 06 verification-before-completion › 07 finishing-a-development-branch

COMO COMEÇAR

```
# só as skills do fluxo acima (no Windows, numa linha só)
$ npx skills add obra/superpowers -g -a github-copilot \
    -s brainstorming writing-plans using-git-worktrees \
    subagent-driven-development test-driven-development \
    requesting-code-review verification-before-completion \
    finishing-a-development-branch
```

Depois, dentro do **copilot**: **/skills list**

skills.sh/obra · github.com/obra/superpowers

Confiável = evidência observável.

AUTOR E CATÁLOGO

UM CATÁLOGO DE MÉTODOS

obra

Jesse Vincent publica skills que codificam práticas de engenharia sênior para agentes.

skills.sh

Diretório aberto de skills: você encontra, instala e usa no seu agente.

OUTROS PACOTES DO AUTOR

episodic-memory	o agente lembra de conversas anteriores
the-elements-of-style	escrita clara e concisa
superpowers-lab	experimentos: tmux, funções duplicadas…
superpowers-chrome	navegar na web pelo Chrome

O principal é o **superpowers** →

SKILLS POR PLATAFORMA · 1 DE 3

iOS · SWIFT

swiftui-pro

twostraws/swiftui-agent-skill comunidade

Revisa código SwiftUI: APIs modernas, manutenção e performance.

swiftui-expert-skill

avdlee/swiftui-agent-skill comunidade

Escreve e refatora SwiftUI: estado com `@Observable`, composição de views e listas.

swift-concurrency

avdlee/swift-concurrency-agent-skill comunidade

Diagnostica concorrência, troca callbacks por async/await e guia a migração para o Swift 6.

swift-testing-pro

twostraws/swift-testing-agent-skill comunidade

Escreve e revisa testes com o framework Swift Testing.

ios-accessibility

dpearson2699/swift-ios-skills comunidade

Constrói e audita acessibilidade: VoiceOver, Voice Control e Dynamic Type.

COMBINA COM O SUPERPOWERS

`swift-testing-pro` + `test-driven-development`: o teste vem primeiro, já com as APIs do Swift Testing.

No Copilot CLI: copie a pasta da skill para **`.github/skills/`** ou **`~/.copilot/skills/`**

Revise o **`SKILL.md`** antes de instalar.

SKILLS POR PLATAFORMA · 2 DE 3

ANDROID · KOTLIN

migrate-xml-views-to-jetpack-compose

android/skills oficial

Roteiro guiado para migrar telas em XML para Jetpack Compose.

navigation-3

android/skills oficial

Instala e migra para o Navigation 3: deep links, múltiplas pilhas e diálogos.

edge-to-edge

android/skills oficial

Adapta apps Compose para a tela de borda a borda e corrige os problemas comuns.

kotlin-tooling-java-to-kotlin

kotlin/kotlin-agent-skills JetBrains

Converte código Java em Kotlin idiomático.

kotlin-coroutines-flows

affaan-m/ecc comunidade

Coroutines e Flow: concorrência estruturada, StateFlow, erros e testes.

COMBINA COM O SUPERPOWERS

`migrate-xml-views-to-jetpack-compose` + `writing-plans`: a migração vira um plano tela a tela.

No Copilot CLI: copie a pasta da skill para **`.github/skills/`** ou **`~/.copilot/skills/`**

Revise o **`SKILL.md`** antes de instalar.

SKILLS POR PLATAFORMA · 3 DE 3

BACKEND · JAVA

java-springboot

github/awesome-copilot GitHub

Boas práticas para aplicações Spring Boot.

create-spring-boot-java-project

github/awesome-copilot GitHub

Gera o esqueleto de um projeto Spring Boot em Java.

java-junit

github/awesome-copilot GitHub

Testes com JUnit 5, incluindo testes orientados a dados.

springboot-security

affaan-m/ecc comunidade

Spring Security: autenticação, autorização, validação, CSRF, segredos e rate limiting.

springboot-tdd

affaan-m/ecc comunidade

TDD com JUnit 5, Mockito, MockMvc, Testcontainers e JaCoCo.

COMBINA COM O SUPERPOWERS

`springboot-tdd` + `test-driven-development`: o vermelho → verde de hoje, com JUnit 5 e Mockito.

No Copilot CLI: copie a pasta da skill para **`.github/skills/`** ou **`~/.copilot/skills/`**

Revise o **`SKILL.md`** antes de instalar.